PART II: RELATIONAL MODEL, ALGEBRA & SQL • Chapter 6

Relational Algebra and Relational Calculus

CHAPTER 6 ROADMAP & LEARNING OBJECTIVES

Why This Matters: *Every time you run a SQL query in PostgreSQL, Oracle, or MySQL, the Query Compiler immediately translates your declarative SQL into a Relational Algebra expression tree so the Query Optimizer can apply algebraic equivalence rules! Mastering Relational Algebra and Relational Calculus gives you X-ray vision into query optimization and is a guaranteed multi-question topic in GATE CS and university exams.*

Prerequisites: Chapter 5 (Mathematical Relations, Set Semantics, Keys, and Foreign Keys).

Learning Objectives (What You Will Master in This Chapter):

- Contrast Procedural Relational Algebra with Declarative Relational Calculus (TRC and DRC), and explain the Closure Property of Relational Algebra.
- Distinguish the 6 Fundamental Operators {σ, π, ∪, −, ×, ρ} from the Derived Operators {∩, ⋈_θ, ⋈, ⟕, ▷, ÷} and express every derived operator using only fundamental operators.
- Master Unary operators (Selection σ, Projection π, Rename ρ), Set-Theoretic operators (∪, −, ∩, ×), and write the classic GATE query to find the Maximum and Second-Highest value without aggregate functions!
- Analyze Theta Join, Equijoin, Natural Join (including edge cases $R \cap S = \emptyset$ and $R = S$), Outer Joins, Semijoin (⋉), Antijoin (▷), and the Division Operator (÷) for Universal Quantification ("For All").
- Memorize and apply the 12-row GATE CS Master Table of Minimum and Maximum Cardinality Bounds for all relational operations.
- Write safe Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC) formulas and apply Codd's Theorem.

Contents

6 Relational Algebra and Relational Calculus 1

6.1 Why Learn Relational Algebra and Calculus? (Procedural vs. Declarative) 3

6.1.1 The Closure Property & The 6 Fundamental vs. 6 Derived Operators (Table 6.1) 3

6.2 Unary Operations: Selection (σ), Projection (π), and Rename (ρ) 4

6.2.1 Algebraic Properties of σ and π (Commutativity, Cascade & Duplicate Elimination) 4

6.3 Set-Theoretic Operations and Cartesian Product (∪, −, ∩, ×) 5

6.3.1 Union Compatibility & Worked Examples 6.1–6.6 (Finding Maximum Without MAX()!) 5

6.4 Join Operations: Theta, Equijoin, Natural, Outer, Semijoin (⋉) & Antijoin (▷) 7

6.4.1 Natural Join Edge Cases & Semijoin / Antijoin identities 7

6.5 The Division Operator (÷): Universal Quantification ("For All") 9

6.5.1 Worked Example 6.7: Expressing R ÷ S Using Only {π, ×, −} 9

6.6 GATE CS Masterclass: Minimum and Maximum Cardinality Bounds (Table 6.2) 10

6.6.1 Worked Example 6.8: Solving GATE Cardinality Bound Problems 11

6.7 Extended Relational Algebra (γ, τ, δ) and Query Optimizer Expression Trees 12

6.8 Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC) 13

6.8.1 Free vs. Bound Variables, Universal Quantification (∀), Safe Expressions & Codd's Theorem 13

6.9 Master Gallery: Side-by-Side Algebra, TRC, and DRC Worked Examples 15

6.10 Chapter 6 Summary & One-Page Rapid-Revision Cheat Sheet 16

6.11 Chapter 6 Exam, GATE & Placement Question Bank (Parts A–H) 18

6.12 Chapter 6 Complete Answer Key & Worked Solutions 22

6.1 Why Learn Relational Algebra and Calculus? (Procedural vs. Declarative)

Formal relational query languages fall into two paradigms: Relational Algebra (Procedural—specifies a step-by-step sequence of operator applications to construct the result) and Relational Calculus (Declarative—specifies the logical predicate that result tuples must satisfy).

6.1.1 The Closure Property & Fundamental vs. Derived Operators

Closure Property: Both the inputs and the output of every Relational Algebra operator are Relations (Sets of Tuples)! Because the output of one operator is a valid relation, operators can be nested arbitrarily into complex Algebraic Expression Trees.

Table 6.1: Complete Reference of the 6 Fundamental Operators and 6 Derived Operators in Relational Algebra

Category	Operator & Symbol	Syntax & Exact Expressive Equivalence
1. Fundamental (Unary)	Selection (σ)	σ_p(R) — Filters rows satisfying predicate p. Degree = deg(R); 0 <= |σ_p(R)| <= |R|.
2. Fundamental (Unary)	Projection (π)	π_L(R) — Keeps columns in list L and automatically eliminates duplicate tuples!
3. Fundamental (Unary)	Rename (ρ)	ρ_{S(B1..Bn)}(R) — Renames relation R to S and/or aliases attributes (essential for self-joins).
4. Fundamental (Set)	Union (∪)	R ∪ S — Tuples in R or S (requires Union Compatibility; eliminates duplicates).
5. Fundamental (Set)	Set Difference (−)	R − S — Tuples present in R but NOT in S (requires Union Compatibility).
6. Fundamental (Binary)	Cartesian Product (×)	R × S — Pairs every tuple of R with every tuple of S. deg = deg(R)+deg(S); |R × S| = |R|·|S|.
Derived Operator 1	Intersection (∩)	R ∩ S ≡ R − (R − S) [Expressed using Set Difference only!]
Derived Operator 2	Theta Join (⋈_θ)	R ⋈_θ S ≡ σ_θ(R × S) [Cartesian Product followed by Selection]
Derived Operator 3	Natural Join (⋈)	R ⋈ S ≡ π_{unique_attrs}(σ_{R.A1=S.A1 ∧ ...}(R × S)) [Equijoin on common columns]
Derived Operator 4	Left Semijoin (⋉)	R ⋉ S ≡ π_{attrs(R)}(R ⋈ S) [Tuples of R that have at least one match in S]
Derived Operator 5	Antijoin (▷ or −⋉)	R ▷ S ≡ R − (R ⋉ S) ≡ R − π_{attrs(R)}(R ⋈ S) [Tuples of R with ZERO matches in S]
Derived Operator 6	Division (÷)	R(X,Y) ÷ S(Y) ≡ π_X(R) − π_X((π_X(R) × S) − R) ["For All" Universal Quantification!]

6.2 Unary Operations: Selection (σ), Projection (π), and Rename (ρ)

`Selection σ_{CGPA > 8.0}(Student)`	`Slices HORIZONTALLY: Keeps all 5 columns, returns only rows where CGPA > 8.0 (Asha, Meera).`
`Projection π_{City}(Student)`	`Slices VERTICALLY: Keeps only City column AND removes duplicate "Patna" -> {"Patna", "Delhi", "Jaipur"} (3 rows, not 4!).`
`Composed Pipeline π_{Name, CGPA}(σ_{DeptID = 10}(Student))`	`First filters CSE students (Asha, Ravi), then projects (Name, CGPA).`

Figure 6.1: Visualizing Selection σ (Horizontal Row Slicing), Projection π (Vertical Column Slicing + Deduplication), and Pipelining π(σ(R)).

6.2.1 Key Algebraic Properties of σ and π (Used by Query Optimizers)

1. Selection Is Commutative: σ_{p1}(σ_{p2}(R)) ≡ σ_{p2}(σ_{p1}(R)) ≡ σ_{p1 ∧ p2}(R).
2. Cascade of Projections: π_{L1}(π_{L2}(R)) ≡ π_{L1}(R) provided L1 ⊆ L2. Note: Projection is NOT commutative!
3. Pushing Selection Below Projection: σ_p(π_L(R)) ≡ π_L(σ_p(R)) ONLY IF every attribute referenced in predicate p is present in the projection list L!
4. Duplicate Elimination in Pure π vs. SQL SELECT: Pure Relational Algebra π_L(R) ALWAYS eliminates duplicate tuples (Set semantics) and therefore corresponds to SQL SELECT DISTINCT L FROM R, NOT plain SELECT L FROM R!

6.3 Set-Theoretic Operations and Cartesian Product (∪, −, ∩, ×)

6.3.1 Union Compatibility (Type Compatibility) Requirement

For R ∪ S, R − S, and R ∩ S to be valid, relations R and S MUST be Union-Compatible: (1) They must have the exact same Degree deg(R) = deg(S), and (2) For every column position i (1 <= i <= n), dom(A_i) of R must be compatible with dom(B_i) of S.

- Commutativity & Associativity: Union (∪) and Intersection (∩) are both commutative (R ∪ S = S ∪ R) and associative. Set Difference (−) is NEITHER commutative (R − S ≠ S − R) NOR associative!
- Cartesian Product (×): Does NOT require Union Compatibility. If R has n1 columns and m1 rows, and S has n2 columns and m2 rows, then R × S has (n1 + n2) columns and (m1 × m2) rows.

6.3.2 Worked Examples 6.1–6.6: Finding Maximum & Second-Highest Without Aggregation!

GATE CLASSIC (Worked Example 6.5) | Finding the Maximum CGPA in Pure Relational Algebra (Without MAX()!)

How do you find the highest CGPA in Student(StudentID, Name, CGPA) when standard Relational Algebra has NO aggregate functions (MAX, MIN, COUNT)?

Key Insight: We CAN easily find all CGPAs that are NOT the maximum—namely, any CGPA that is strictly smaller than some other student's CGPA in a self-Cartesian product! Subtracting the "non-maximum" CGPAs from "all CGPAs" leaves the Maximum CGPA!

Step 1 (All CGPAs): A = π_{CGPA}(Student)
Step 2 (Strictly Smaller / Non-Max CGPAs via Rename ρ): B = π_{s1.CGPA}(σ_{s1.CGPA < s2.CGPA}(ρ_{s1}(Student) × ρ_{s2}(Student)))
Step 3 (Maximum CGPA = All − Non-Max):
Max_CGPA = π_{CGPA}(Student) − π_{s1.CGPA}(σ_{s1.CGPA < s2.CGPA}(ρ_{s1}(Student) × ρ_{s2}(Student)))

Extension (Worked Example 6.6 — Finding the Second-Highest CGPA):
Let Student_Rem = σ_{CGPA < Max_CGPA}(Student) (or subtract Max_CGPA from all CGPAs: C_rem = A − Max_CGPA), and apply the exact same "All − Strictly Smaller" pattern to C_rem!

6.4 Join Operations: Theta, Equijoin, Natural, Outer, Semijoin & Antijoin

6.4.1 Theta Join (⋈_θ), Equijoin, and Natural Join (⋈)

1. Theta Join ($R \bowtie_\theta S \equiv \sigma_\theta(R \times S)$): Combines tuples from R and S satisfying an arbitrary comparison condition θ. Retains all deg(R) + deg(S) columns.
2. Equijoin: A Theta Join where the condition θ consists exclusively of equality (=) comparisons. Still retains duplicate join columns (both R.DeptID and S.DeptID appear).
3. Natural Join ($R \bowtie S$): An Equijoin on ALL attributes that have the SAME NAME in both R and S, followed by projecting out the duplicate common columns so each common attribute appears only once! Degree of $R \bowtie S$ = deg(R) + deg(S) − |attrs(R) ∩ attrs(S)|.

TWO FAMOUS GATE TRAPS ON NATURAL JOIN (R ⋈ S)

Trap 1 (Disjoint Schemas — Zero Common Attributes): If attrs(R) ∩ attrs(S) = ∅ (R and S share NO column names), what is R ⋈ S?
• Answer: It degenerates into the full Cartesian Product R × S (NOT an empty set)!

Trap 2 (Identical Schemas — All Attributes in Common): If attrs(R) = attrs(S) (R and S have the exact same columns), what is R ⋈ S?
• Answer: It equals the Set Intersection R ∩ S!

6.4.2 Outer Joins, Left Semijoin (⋉), and Antijoin (▷)

- Left / Right / Full Outer Join (⟕, ⟖, ⟗): Preserves "dangling tuples" (rows that have no match in the other table) by padding missing attributes with NULL.
- Left Semijoin ($R \ltimes S \equiv \pi_{attrs(R)}(R \bowtie S)$): Returns tuples of R that match at least one tuple in S, without duplicating R tuples even if S has 50 matching rows! Used in distributed query optimization and SQL WHERE EXISTS.
- Antijoin ($R \triangleright S \equiv R - (R \ltimes S)$): Returns tuples of R that match ZERO tuples in S (equivalent to SQL WHERE NOT EXISTS). Note: $R \ltimes S \neq S \ltimes R$!

Input R = Student {(1,Asha), (2,Ravi), (4,Kabir)}	Input S = Enrollment {(1,CS301), (1,CS201), (2,CS301)} [Kabir has 0 enrollments; Asha has 2]
Inner Natural Join (R ⋈ S)	Returns 3 rows: (1,Asha,CS301), (1,Asha,CS201), (2,Ravi,CS301) [Kabir dropped; Asha duplicated]
Left Outer Join (R ⟕ S)	Returns 4 rows: (1,Asha,CS301), (1,Asha,CS201), (2,Ravi,CS301), (4,Kabir,NULL)
Left Semijoin (R ⋉ S) vs. Antijoin (R ▷ S)	R ⋉ S = {(1,Asha), (2,Ravi)} (Asha appears ONCE!) | R ▷ S = {(4,Kabir)} (Unenrolled students!)

Figure 6.2: Tuple-by-Tuple Behavior of Inner Join, Left Outer Join, Semijoin (⋉), and Antijoin (▷) on Student and Enrollment.

6.5 The Division Operator (÷): Universal Quantification ("For All")

Whenever an English query contains the words "Find students who have taken ALL courses offered by CSE" or "Find suppliers who supply EVERY part," the Relational Algebra operator is Division (R ÷ S)!

CONCEPT | Formal Definition of Division R(X, Y) ÷ S(Y)

Let R(X, Y) be a relation over attribute sets X and Y, and let S(Y) be a relation over Y. The division R ÷ S returns a relation over attribute set X containing every x-value such that for EVERY tuple y ∈ S, the pair (x, y) is present in R:

$R \div S = \{ t[X] \mid t \in R \land \forall s \in S, \langle t[X], s[Y] \rangle \in R \}$

Why is it called Division? Because it is the inverse of Cartesian Product: $(R \times S) \div S = R$, and for any $Q = R \div S$, Q is the largest relation satisfying $Q \times S \subseteq R$ (just like integer division q = a ÷ b satisfies q × b <= a)!

Dividend R(StudentID, CourseID):	{ (1, CS301), (1, CS201), (1, CS101), (2, CS301), (3, CS301), (3, CS201) }
Divisor S(CourseID):	{ (CS301), (CS201) } <--- Must match BOTH CS301 and CS201
Quotient Result R ÷ S (over StudentID):	{ (1), (3) } [Student 1 has {CS301,CS201,CS101} ⊇ S; Student 3 has {CS301,CS201} ⊇ S; Student 2 misses CS201]

Figure 6.3: Visual Trace of Division R(StudentID, CourseID) ÷ S(CourseID) to find Students Enrolled in ALL Required Courses.

6.5.1 Worked Example 6.7: Expressing Division (R ÷ S) Using Only {π, ×, −}

WORKED EXAMPLE 6.7 | 3-Step Derivation of R(X, Y) ÷ S(Y) Using Fundamental Operators

How do we express R(X, Y) ÷ S(Y) using only Projection (π), Cartesian Product (×), and Set Difference (−)?

Step 1 (All Candidate X-Values and All Ideal (X, Y) Pairs):
• Candidates = π_X(R)
• Ideal_Pairs = π_X(R) × S (Every candidate student paired with EVERY required course in S).

Step 2 (Find the "Disqualified" X-Values that Are MISSING at Least One Y in S):
• Missing_Pairs = (π_X(R) × S) − R (Pairs that SHOULD exist if x took all courses, but are missing from R).
• Disqualified_X = π_X((π_X(R) × S) − R) (Students who missed at least one course in S).

Step 3 (Subtract Disqualified X-Values from All Candidate X-Values):
R ÷ S ≡ π_X(R) − π_X((π_X(R) × S) − R)

Notice how this double Set Difference (− ... −) mirrors the SQL double NOT EXISTS pattern for "For All" queries!

6.6 GATE CS Masterclass: Minimum and Maximum Cardinality Bounds

Let relation R have m tuples (|R| = m) and relation S have n tuples (|S| = n). Memorize this 12-row GATE CS Master Table of exact minimum and maximum output cardinalities:

Table 6.2: GATE CS Master Reference Table — Minimum and Maximum Cardinality Bounds (|R| = m, |S| = n)

Relational Operation	Minimum Cardinality (Min Rows)	Maximum Cardinality (Max Rows)	Condition for Min / Max
1. Selection σ_p(R)	0	m	Min when 0 rows match p; Max when all m rows match p.
2. Projection π_L(R) (Non-Key L, m >= 1)	1	m	Min = 1 if all m rows share the exact same value on L; Max = m if all distinct (or L contains a Super Key -> always m!).
3. Union R ∪ S	max(m, n)	m + n	Min when one relation is a subset of the other; Max when R and S are completely disjoint.
4. Intersection R ∩ S	0	min(m, n)	Min when R and S are disjoint; Max when the smaller relation is a subset of the larger.
5. Set Difference R − S	max(0, m − n)	m	Min when as many tuples of S as possible belong to R; Max when R and S are disjoint.
6. Cartesian Product R × S	m × n	m × n	Always exactly m × n tuples.
7. Natural / Theta Join R ⋈ S (No Key/FK)	0	m × n	Min = 0 (no matching join values); Max = m × n (all tuples of R and S share the same join value!).
8. Join R ⋈ S where Join Attr is PK of S	0	m	Each of the m tuples in R can match at most 1 tuple in S (if FK is nullable/unconstrained).
9. Join R ⋈ S where R.FK (NOT NULL) -> S.PK	m	m	Every tuple of R MUST match exactly 1 tuple of S -> output is ALWAYS exactly m!
10. Left Outer Join R ⟕ S (General)	m	m × n	All m rows of R are preserved at minimum; Max = m × n if all rows match every row of S.
11. Full Outer Join R ⟗ S (General)	max(m, n)	m × n (or m + n if 0 matches)	If 0 rows match, yields m + n padded rows; if all m rows match all n rows, yields m × n rows.
12. Division R(X,Y) ÷ S(Y) (|S| = n >= 1)	0	floor(m / n)	Each winning x-value requires at least n distinct tuples in R -> at most floor(m / n) winners!

6.6.1 Worked Example 6.8: Solving GATE Cardinality Bound Problems

WORKED EXAMPLE 6.8 | Three High-Yield GATE Cardinality Calculations

Case 1: Let |R| = 40 and |S| = 30 (Union-Compatible). Find the Min and Max cardinality of R ∪ S, R ∩ S, and R − S.
• R ∪ S: Min = max(40, 30) = 40; Max = 40 + 30 = 70.
• R ∩ S: Min = 0; Max = min(40, 30) = 30.
• R − S: Min = max(0, 40 − 30) = 10; Max = 40.

Case 2 (Foreign Key Join): Student has m = 500 rows and Department has n = 12 rows. Student.DeptID is a Foreign Key referencing Department.DeptID:
• Subcase 2A (DeptID is NOT NULL): Every student matches exactly 1 department -> Min = 500, Max = 500!
• Subcase 2B (DeptID allows NULLs): If all 500 students have DeptID = NULL, 0 rows match -> Min = 0, Max = 500!

Case 3 (Division Bound): Enrollment(StudentID, CourseID) has m = 85 rows, and Required_Courses(CourseID) has n = 6 rows. What is the maximum number of rows in Enrollment ÷ Required_Courses?
• Max rows = floor(85 / 6) = 14 students!

6.7 Extended Relational Algebra (γ, τ, δ) and Expression Trees

To model modern SQL features (GROUP BY, aggregate functions, ORDER BY, and bag deduplication), database engines use three Extended Relational Algebra operators:

1. Grouping and Aggregation Operator (_{G1, G2} γ _{F1(A1), F2(A2)} (R)): Partition tuples of R into groups sharing the same values on grouping attributes (G1, G2), and compute aggregate functions (COUNT, SUM, AVG, MIN, MAX) per group. If the left subscript G is omitted, the entire relation is treated as a single group.
2. Sorting Operator (τ_{A1 ASC, A2 DESC}(R)): Orders tuples for final display or Merge-Join pipelines.
3. Duplicate Elimination Operator (δ(R)): Converts a Bag (Multiset) into a Set (models SQL DISTINCT).

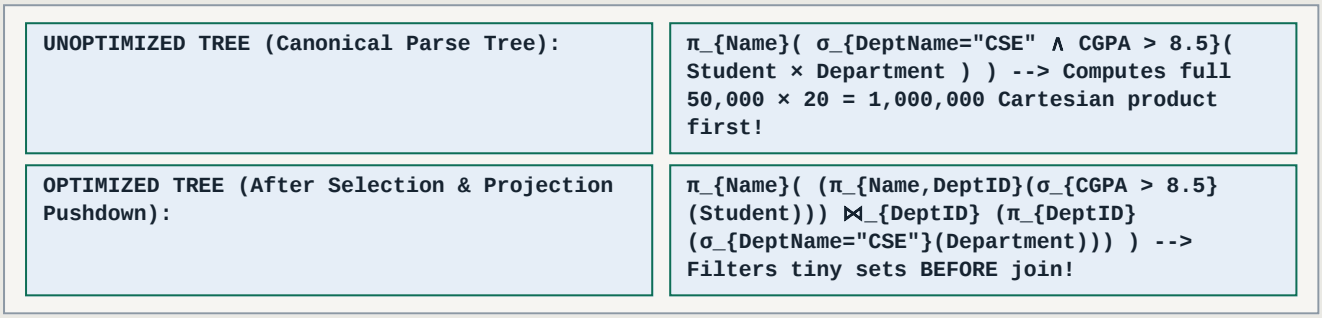


Figure 6.4: Query Optimizer Heuristic Pushdown — Unoptimized Algebra Tree vs. Optimized Tree (Selection & Projection Pushdown).

6.8 Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC)

Unlike Procedural Relational Algebra (which specifies how to apply operators in order), Relational Calculus is a Declarative language based on First-Order Predicate Logic—the direct theoretical inspiration for SQL!

6.8.1 Tuple Relational Calculus (TRC): { t | P(t) }

In TRC, variables range over Tuples (rows). A query has the form { t | P(t) }, meaning "the set of all tuples t such that predicate P(t) is true":

1. Free vs. Bound Tuple Variables: The variable t to the left of the bar "|" is the only FREE variable! Every other tuple variable introduced inside P(t) MUST be bound by an Existential Quantifier (∃s ∈ R) or a Universal Quantifier (∀s ∈ R).
2. Logical Equivalence for Universal Quantification (∀) and Implication (⇒): In TRC/DRC and GATE exams, remember:
   • P ⇒ Q ≡ ¬P ∨ Q (Material Implication)
   • ∀x (P(x)) ≡ ¬∃x (¬P(x)) (De Morgan's Quantifier Law — "All courses are passed" ≡ "There does NOT exist a course that is NOT passed").
3. Safe vs. Unsafe TRC Expressions (Domain Independence): An expression like { t | ¬Student(t) } ("all tuples in the universe that are NOT students") is UNSAFE because it returns an infinite set! A TRC expression is SAFE iff every value in the result tuples comes from the Active Domain DOM(P) of constants and relations appearing in P.
4. Domain Relational Calculus (DRC): Variables range over individual Attribute Domains (columns) rather than whole tuples: { <x1, x2, ..., xn> | P(x1, x2, ..., xn) }. Mirrors QBE (Query-By-Example).

CODD'S THEOREM (1972) | Relational Completeness

E.F. Codd proved that Relational Algebra (using the 6 fundamental operators), Safe Tuple Relational Calculus (TRC), and Safe Domain Relational Calculus (DRC) have the EXACT SAME expressive power! Any query expressible in one can be translated into the other two. Any query language at least as expressive as these three is called Relationally Complete.

6.9 Master Gallery: Side-by-Side Algebra, TRC, and DRC Worked Examples

Side-by-Side Translation of Core Queries Across Relational Algebra, TRC, and DRC

English Query on UniVerseDB	Relational Algebra Expression	Tuple Relational Calculus (TRC) & Domain Calculus (DRC)
1. Names of students in Dept 10 with CGPA > 8.5	π_{Name}(σ_{DeptID=10 ∧ CGPA>8.5}(Student))	TRC: { t.Name | Student(t) ∧ t.DeptID = 10 ∧ t.CGPA > 8.5 } DRC: { <n> | ∃id,d,c,city (Student(id,n,d,c,city) ∧ d=10 ∧ c>8.5) }
2. Names of students enrolled in course "CS301"	π_{Name}(Student ⋈ σ_{CourseID="CS301"}(Enrollment))	TRC: { t.Name | Student(t) ∧ ∃e (Enrollment(e) ∧ e.StudentID = t.StudentID ∧ e.CourseID = "CS301") }
3. Names of students NOT enrolled in ANY course (Antijoin)	π_{Name}(Student ▷ Enrollment) ≡ π_{Name}(Student) − π_{Name}(Student ⋈ Enrollment)	TRC: { t.Name | Student(t) ∧ ¬∃e (Enrollment(e) ∧ e.StudentID = t.StudentID) }
4. Students enrolled in ALL courses of Dept 10 (Division ÷)	π_{StudentID,CourseID}(Enrollment) ÷ π_{CourseID}(σ_{DeptID=10}(Course))	TRC: { t.StudentID | Student(t) ∧ ∀c ((Course(c) ∧ c.DeptID=10) ⇒ ∃e (Enrollment(e) ∧ e.StudentID=t.StudentID ∧ e.CourseID=c.CourseID)) }

EXAM CHECKPOINT | Why Universal Quantification Always Uses Implication (⇒), Never AND (∧)!

In Query 4 ("for all courses c in Dept 10, the student is enrolled in c"), students often write:
∀c (Course(c) ∧ c.DeptID = 10 ∧ ∃e (...)) <-- WRONG!
Why is AND wrong with ∀c? Because ∀c requires the formula to be true for EVERY course c in the database—and as soon as c is an ECE course (DeptID = 20), (c.DeptID = 10) is FALSE, making the entire AND expression FALSE! Always use:
• Existential (∃): ∃c (Condition(c) ∧ Action(c))
• Universal (∀): ∀c (Condition(c) ⇒ Action(c))

6.10 Chapter 6 Summary & One-Page Rapid-Revision Cheat Sheet

Chapter 6 Rapid-Revision Formula & Equivalence Sheet

Topic / Operator	Core Exam Formula / Equivalence
6 Fundamental Operators	{ σ (Select), π (Project), ∪ (Union), − (Set Diff), × (Cartesian Product), ρ (Rename) }.
6 Derived Operators	{ ∩ (Intersect), ⋈_θ (Theta Join), ⋈ (Natural Join), ⋉ (Semijoin), ▷ (Antijoin), ÷ (Division) }.
Intersection Identity	R ∩ S ≡ R − (R − S)
Division Identity (R ÷ S)	R(X,Y) ÷ S(Y) ≡ π_X(R) − π_X((π_X(R) × S) − R)
Finding Max Value in Algebra	π_A(R) − π_{r1.A}(σ_{r1.A < r2.A}(ρ_{r1}(R) × ρ_{r2}(R)))
Natural Join Edge Cases	If attrs(R) ∩ attrs(S) = ∅ => R ⋈ S = R × S. If attrs(R) = attrs(S) => R ⋈ S = R ∩ S.
Cardinality of R ⋈ S (FK -> PK)	If R.FK (NOT NULL) references S.PK: Min = |R|, Max = |R|. If R.FK is nullable: Min = 0, Max = |R|.
TRC Universal Quantification	∀x (P(x) ⇒ Q(x)) ≡ ∀x (¬P(x) ∨ Q(x)) ≡ ¬∃x (P(x) ∧ ¬Q(x)).
Safe TRC / DRC Expression	Guarantees a finite output set whose values come exclusively from the Active Domain DOM(P).

Student Revision & Lecture Notes — Chapter 6

Use this page to practice writing the Division identity, the Maximum-without-aggregation expression, and the 12-row GATE Cardinality Bound formulas.

Active Recall Practice Grid — Chapter 6

Question Prompt	Your Recall Formula
Write R ∩ S using only fundamental operators	R − (R − S)
Write R(A,B) ÷ S(B) using only {π, ×, −}	π_A(R) − π_A((π_A(R) × S) − R)
Min & Max rows of R(40) − S(15)	Min = max(0, 40 − 15) = 25; Max = 40.
Why must ∀x use (P(x) ⇒ Q(x)) while ∃x uses (P(x) ∧ Q(x))?	∀x (P ∧ Q) fails as soon as any tuple does not satisfy P; P ⇒ Q is vacuously true when P is false!

6.11 Chapter 6 Exam, GATE & Placement Question Bank

Part A: 10 Multiple-Choice Questions (MCQs)

1. Q1. Which of the following sets of Relational Algebra operators is a minimal complete (fundamental) set from which all other standard relational algebra operators can be derived?
   (A) {σ, π, ∪, ∩, ×} (B) {σ, π, ∪, −, ×, ρ} (C) {σ, π, ⋈, ÷} (D) {σ, π, ∪, ÷}
2. Q2. Which of the following Relational Algebra operators is NOT commutative?
   (A) Selection: σ_p1(σ_p2(R)) (B) Natural Join: R ⋈ S (C) Set Difference: R − S (D) Set Intersection: R ∩ S
3. Q3. Let R(A, B) and S(C, D) be two relations with no common attribute names (attrs(R) ∩ attrs(S) = ∅). If |R| = 10 and |S| = 20, how many tuples are in the Natural Join R ⋈ S?
   (A) 0 (B) 10 (C) 20 (D) 200
4. Q4. Which Relational Algebra expression correctly represents the Division operation R(X, Y) ÷ S(Y)?
   (A) π_X(R) − π_X(R × S) (B) π_X(R) − π_X((π_X(R) × S) − R) (C) π_X(R × S) − π_X(R) (D) R − (π_X(R) × S)
5. Q5. Let R have m = 50 tuples and S have n = 35 tuples, where R and S are union-compatible. What are the minimum and maximum possible number of tuples in R − S?
   (A) 0 and 50 (B) 15 and 50 (C) 15 and 35 (D) 0 and 15

Part A: Multiple-Choice Questions (Continued — Q6 to Q10)

- 6. Q6. What does the Relational Algebra expression π_A(R) − π_{r1.A}(σ_{r1.A < r2.A}(ρ_{r1}(R) × ρ_{r2}(R))) compute?
  (A) The minimum value of attribute A in R (B) The maximum value of attribute A in R (C) All duplicate values of A in R (D) An empty set
- 7. Q7. For any two relations R and S, the Left Semijoin R ⋉ S is defined as:
  (A) π_{attrs(S)}(R ⋈ S) (B) π_{attrs(R)}(R ⋈ S) (C) R − π_{attrs(R)}(R ⋈ S) (D) R ⋊ S
- 8. Q8. Which of the following Tuple Relational Calculus (TRC) expressions is UNSAFE (not domain-independent)?
  (A) { t | Student(t) ∧ t.CGPA > 9.0 } (B) { t | ¬Student(t) } (C) { t | Student(t) ∧ ¬∃e (Enrollment(e) ∧ e.StudentID = t.StudentID) } (D) { t | Department(t) }
- 9. Q9. According to Codd's Theorem, a relational query language is said to be Relationally Complete if it is at least as expressive as:
  (A) CODASYL Network Pointers (B) Safe Tuple Relational Calculus / Standard Relational Algebra (C) Regular Expressions (D) Key-Value Hash Lookups
- 10. Q10. Let R(A, B, C) have 100 tuples, where {A, B} is a Candidate Key. How many tuples are returned by π_{A, B}(R)?
  (A) Between 1 and 100 (B) Exactly 100 (C) At most 50 (D) 0

Part B: 5 Short-Answer Questions (2–3 Marks Semester Exam Style)

1. B1. Explain the Closure Property of Relational Algebra and why it is essential for query optimization.
2. B2. State the two conditions required for two relations R and S to be Union-Compatible.
3. B3. Compare Theta Join, Equijoin, and Natural Join with an example, noting the degree (column count) of each.
4. B4. Why is Projection π_L(R) in Relational Algebra NOT identical to SELECT L FROM R in SQL?
5. B5. Define a Safe TRC expression and explain why { t | ¬(t ∈ Student) } is unsafe.

Part C: 3 Long-Answer / Essay Questions (5–10 Marks Semester Exam Style)

1. C1. List the 6 Fundamental Operators and 6 Derived Operators of Relational Algebra. Show step-by-step how Intersection (∩), Theta Join (⋈_θ), Semijoin (⋉), Antijoin (▷), and Division (÷) are rewritten using only the 6 fundamental operators.
2. C2. Explain the Division Operator (R ÷ S) in detail with a full numerical trace on Enrollment(StudentID, CourseID) and CoreCourses(CourseID). Derive its fundamental identity and show its equivalent SQL query.
3. C3. Compare Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC) with Relational Algebra. Write 4 UniVerseDB queries (Filter, Join, Antijoin, and Universal Quantification) across all three formalisms.

Part D: 4 Algebraic Expression & Cardinality Problems

1. D1. (Writing 4 UniVerseDB Queries in Relational Algebra): Given Student(_SID_, Name, DeptID, CGPA), Course(_CID_, Title, DeptID), and Enrollment(_SID_, _CID_, Grade), write Relational Algebra expressions for:
   (a) Names of students who earned Grade "A+" in course "CS301".
   (b) Names of students who have enrolled in at least one course offered by DeptID = 10 AND at least one course offered by DeptID = 20.
   (c) Names of students who have NEVER received Grade "F" in any course.
   (d) StudentIDs of students who have taken EVERY course offered by DeptID = 10.
2. D2. (Finding Minimum and Maximum Without Aggregate Operators): Write pure Relational Algebra expressions to find: (a) the lowest (minimum) CGPA in Student, and (b) the names of students who hold the highest CGPA.
3. D3. (Cardinality Bounds Suite): Let |R| = 120 and |S| = 40. Compute [Min, Max] cardinality for: (i) R ∪ S, (ii) R ∩ S, (iii) R − S, (iv) S − R, (v) R ⋈ S (unconstrained), and (vi) R(X,Y) ÷ S(Y).
4. D4. (Query Tree Optimization): Draw the canonical Relational Algebra tree for SELECT s.Name FROM Student s, Enrollment e WHERE s.SID = e.SID AND e.CID = "CS301" AND s.CGPA > 9.0; and transform it into an optimized tree using heuristic pushdown rules.

Part E: 5 GATE CS Style Questions (MCQ, MSQ, and NAT)

1. E1. [GATE NAT — 2 Marks] Let R(A, B) have 80 tuples and S(B, C) have 25 tuples. Suppose B is the Primary Key of S, and R.B is a Foreign Key referencing S.B that allows NULL values. If exactly 15 tuples in R have B = NULL, the number of tuples in the Natural Join R ⋈ S is _____.
2. E2. [GATE NAT — 2 Marks] Let R(A, B, C) have 200 tuples and S(B, C) have 8 tuples (all distinct). What is the maximum possible number of tuples in the division result R ÷ S? _____.
3. E3. [GATE MCQ — 2 Marks] Consider the Relational Algebra expression:
   E = π_A(R) − π_A((π_A(R) × S) − R)
   where R(A, B) and S(B) are relations. Which of the following TRC queries is equivalent to E?
   (A) { t.A | R(t) ∧ ∃s ∈ S (<t.A, s.B> ∈ R) }
   (B) { t.A | R(t) ∧ ∀s ∈ S (<t.A, s.B> ∈ R) }
   (C) { t.A | R(t) ∧ ∀s ∈ S (<t.A, s.B> ∉ R) }
   (D) { t.A | ¬∃s ∈ S (<t.A, s.B> ∈ R) }
4. E4. [GATE MSQ — 2 Marks] Let R and S be two relations with identical schemas attrs(R) = attrs(S). Which of the following algebraic equivalences hold for ALL instances of R and S?
   (A) R ⋈ S ≡ R ∩ S
   (B) R − (R − S) ≡ R ∩ S
   (C) R ⋉ S ≡ R ∩ S
   (D) π_L(R − S) ≡ π_L(R) − π_L(S) for any proper subset of attributes L ⊂ attrs(R)
5. E5. [GATE NAT — 2 Marks] Let R(A, B) contain the tuples {(1, 10), (1, 20), (2, 10), (2, 30), (3, 10), (3, 20), (3, 30)} and let S(B) contain {(10), (20)}. The number of tuples in R ÷ S is _____.

Part F: 5 Viva / Technical Placement Interview Questions

1. F1. "Why does a DBMS compiler translate SQL into a Relational Algebra tree instead of executing the SQL text directly?"
2. F2. "Why doesn't SQL have a direct DIVIDE BY keyword for Relational Algebra Division (÷), and how do you write Division in SQL?"
3. F3. "Why is π_L(R − S) NOT equivalent to π_L(R) − π_L(S)?"
4. F4. "When does a Query Optimizer use a Left Semijoin (⋉) or Antijoin (▷) physical operator during SQL execution?"
5. F5. "Explain why pushing Selection (σ) and Projection (π) down below Joins (⋈) is the #1 heuristic optimization rule in every RDBMS."

Part G: 3 Hands-On Algebra-to-SQL Labs & Part H: 3 "Spot the Mistake" Challenges

1. Lab 6.1: Implement Relational Division R ÷ S in SQL using two different patterns: (a) Double NOT EXISTS and (b) GROUP BY ... HAVING COUNT(DISTINCT CourseID) = (SELECT COUNT(*) FROM S).
2. Lab 6.2: Implement the Relational Algebra "Maximum CGPA Without MAX()" self-join query in SQL and compare its result with SELECT MAX(CGPA).
3. Lab 6.3: Demonstrate Left Semijoin (WHERE EXISTS) vs. Inner Join on Student and Enrollment to show why Inner Join duplicates students with multiple courses.
4. Mistake 6.1: A student claims that π_{DeptID}(Student − Alumni) = π_{DeptID}(Student) − π_{DeptID}(Alumni).
5. Mistake 6.2: To find students enrolled in BOTH CS301 and CS201, a student writes σ_{CourseID="CS301" ∧ CourseID="CS201"}(Enrollment).
6. Mistake 6.3: A student claims that if R(A,B) and S(C,D) have no common attributes, the Natural Join R ⋈ S returns 0 rows.

6.12 Chapter 6 Complete Answer Key & Worked Solutions

Solutions to Part A: Multiple-Choice Questions (Q1 to Q10)

Answer Key & Detailed Explanations for Part A MCQs

Q#	Ans	Detailed Step-by-Step Explanation
Q1	(B)	The 6 fundamental operators of Relational Algebra are {σ, π, ∪, −, ×, ρ}.
Q2	(C)	Set Difference is NOT commutative: R − S ≠ S − R.
Q3	(D)	When attrs(R) ∩ attrs(S) = ∅, Natural Join R ⋈ S degenerates into R × S -> 10 × 20 = 200 tuples!
Q4	(B)	R(X,Y) ÷ S(Y) ≡ π_X(R) − π_X((π_X(R) × S) − R).
Q5	(B)	For |R| = 50, |S| = 35: Min(R − S) = max(0, 50 − 35) = 15; Max(R − S) = 50.
Q6	(B)	Subtracting all values strictly smaller than some other value from all values leaves the Maximum value!
Q7	(B)	Left Semijoin R ⋉ S projects the Natural Join back onto the attributes of R: π_{attrs(R)}(R ⋈ S).
Q8	(B)	{ t | ¬Student(t) } is UNSAFE because an infinite number of tuples in the universe are not in Student.
Q9	(B)	Codd’s Theorem defines Relational Completeness as equivalence to Safe TRC / Safe DRC / Relational Algebra.
Q10	(B)	Because {A, B} is a Candidate Key, all 100 tuples have distinct (A, B) pairs -> π_{A,B}(R) has 100 tuples!

Solutions to Part B: Short-Answer Questions (B1 to B5)

- Answer B1 (Closure Property): Every Relational Algebra operator takes one or two relations as input and produces a valid relation as output. This allows operators to be composed arbitrarily into expression trees that the Query Optimizer can rewrite using algebraic equivalences.
- Answer B2 (Union Compatibility): Two relations R and S are Union-Compatible iff (1) they have the same degree (number of attributes), and (2) corresponding attributes have compatible data domains.
- Answer B3 (Theta vs. Equijoin vs. Natural Join): Theta Join ($R \bowtie_\theta S$) joins on any condition θ and keeps deg(R)+deg(S) columns; Equijoin uses only equality (=) conditions and still keeps deg(R)+deg(S) columns; Natural Join ($R \bowtie S$) equates all identically named attributes and drops duplicate join columns.
- Answer B4 (Pure π vs. SQL SELECT): Pure Relational Algebra operates on mathematical Sets, so $\pi_L(R)$ automatically removes duplicates (equivalent to SELECT DISTINCT L FROM R). Standard SQL SELECT L FROM R operates on Bags and retains duplicates.
- Answer B5 (Safe TRC Expression): A TRC expression $\{ t \mid P(t) \}$ is safe if every component of every result tuple t is drawn from the finite Active Domain of constants and relation instances appearing in P. $\{ t \mid \neg Student(t) \}$ is unsafe because its output depends on an infinite universe outside the database.

Solutions to Part C (C1–C3) & Part D: Algebraic Expression Problems (D1 & D2)

Worked Solutions D1 | Four UniVerseDB Relational Algebra Queries

(a) Students with Grade "A+" in "CS301":
π_{Name}(Student ⋈ σ_{CID="CS301" ∧ Grade="A+"}(Enrollment))

(b) Students enrolled in at least one Dept 10 course AND at least one Dept 20 course:
π_{Name}(Student ⋈ (π_{SID}(Enrollment ⋈ σ_{DeptID=10}(Course)) ∩ π_{SID}(Enrollment ⋈ σ_{DeptID=20}(Course))))

(c) Students who have NEVER received Grade "F" in any course:
π_{Name}(Student ⋈ (π_{SID}(Student) − π_{SID}(σ_{Grade="F"}(Enrollment))))

(d) StudentIDs of students who have taken EVERY course offered by DeptID = 10 (Division):
π_{SID, CID}(Enrollment) ÷ π_{CID}(σ_{DeptID=10}(Course))

Worked Solutions D2 | Minimum Value & Names of Top-CGPA Students Without Aggregation

(a) Lowest (Minimum) CGPA in Student (All CGPAs minus CGPAs strictly GREATER than some other CGPA):
Min_CGPA = π_{CGPA}(Student) − π_{s1.CGPA}(σ_{s1.CGPA > s2.CGPA}(ρ_{s1}(Student) × ρ_{s2}(Student)))

(b) Names of Students holding the Maximum CGPA:
Max_CGPA = π_{CGPA}(Student) − π_{s1.CGPA}(σ_{s1.CGPA < s2.CGPA}(ρ_{s1}(Student) × ρ_{s2}(Student)))
Result = π_{Name}(Student ⋈ Max_CGPA)

Solutions to Part D (D3 & D4) and Part E: GATE CS Questions (E1 to E5)

Worked Solutions D3 & D4 | Cardinality Bounds Suite & Heuristic Pushdown

Solution D3 (|R| = 120, |S| = 40):
• (i) R ∪ S: [max(120,40), 120+40] = [120, 160]
• (ii) R ∩ S: [0, min(120,40)] = [0, 40]
• (iii) R − S: [max(0, 120−40), 120] = [80, 120]
• (iv) S − R: [max(0, 40−120), 40] = [0, 40]
• (v) R ⋈ S (unconstrained): [0, 120 × 40] = [0, 4800]
• (vi) R(X,Y) ÷ S(Y): [0, floor(120 / 40)] = [0, 3]

Solution D4 (Optimized Algebra Expression after Pushdown):
π_{Name}((π_{SID, Name}(σ_{CGPA > 9.0}(Student))) ⋈_{SID} (π_{SID}(σ_{CID = "CS301"}(Enrollment))))

Step-by-Step GATE CS Solutions for Questions E1 to E5

GATE Q#	Correct Answer	Complete Step-by-Step Derivation
E1 (NAT)	65	Of the 80 tuples in R, 15 have B = NULL (which never match in an inner join). The remaining 80 − 15 = 65 tuples have non-null Foreign Keys referencing S.B (PK of S), so each matches exactly 1 tuple of S -> 65 tuples!
E2 (NAT)	25	Max cardinality of R ÷ S = floor(|R| / |S|) = floor(200 / 8) = 25 tuples.
E3 (MCQ)	(B)	The expression E is the exact fundamental expansion of Division R(A,B) ÷ S(B), which corresponds to Universal Quantification ∀s ∈ S (<t.A, s.B> ∈ R).
E4 (MSQ)	(A), (B), (C)	When attrs(R) = attrs(S), Natural Join, R − (R − S), and Semijoin all equal R ∩ S! (D) is FALSE because projection does NOT distribute over set difference.
E5 (NAT)	2	S = {(10), (20)}. In R: A=1 has {10, 20} (matches S!); A=2 has {10, 30} (misses 20); A=3 has {10, 20, 30} (matches S!). Thus R ÷ S = {(1), (3)} -> 2 tuples!

Solutions to Part F: Viva & Technical Placement Interview Questions (F1 to F5)

- Answer F1 (Why Translate SQL into Relational Algebra): SQL is declarative and allows writing the same logical query in dozens of syntactically different ways. Translating SQL into a canonical Relational Algebra tree lets the Optimizer apply provably correct algebraic rewrite laws (selection pushdown, join reordering via associativity/commutativity) before binding physical operators.
- Answer F2 (Why SQL Lacks DIVIDE BY & Two Ways to Write It): SQL designers omitted DIVIDE BY because real-world queries often need slight variations (e.g., "at least 90% of courses"). In SQL, you write Division either using Double NOT EXISTS (exact mirror of π_X(R) − π_X((π_X(R)×S) − R)) or using GROUP BY StudentID HAVING COUNT(DISTINCT CourseID) = (SELECT COUNT(*) FROM RequiredCourses).
- Answer F3 (Counterexample Proving π_L(R − S) ≠ π_L(R) − π_L(S)): Let R(A,B) = {(1, 10)} and S(A,B) = {(1, 20)}. Then R − S = {(1, 10)}, so π_A(R − S) = {(1)}. However, π_A(R) = {(1)} and π_A(S) = {(1)}, so π_A(R) − π_A(S) = ∅! Projection NEVER distributes over Set Difference!
- Answer F4 (When Optimizers Use Semijoin & Antijoin): Whenever you write WHERE EXISTS (...) or WHERE col IN (SELECT ...), the Optimizer compiles it into a physical Hash Semijoin or Merge Semijoin (⋉) so it stops scanning the inner table on the VERY FIRST match (avoiding duplicate rows!). WHERE NOT EXISTS compiles into an Antijoin (▷).
- Answer F5 (Why Selection/Projection Pushdown Is #1): Joins (⋈) are the most CPU- and memory-intensive operations in a database. Filtering rows (σ) and dropping unused wide columns (π) BEFORE the join shrinks intermediate hash tables and sort buffers by orders of magnitude!

Solutions to Part G: Hands-On Algebra-to-SQL Labs (Labs 6.1 to 6.3)

Complete SQL Implementations of Division (÷), Max Without MAX(), and Semijoin (⋉)

```
-- Lab 6.1 (Method A): Relational Division R ÷ S via Double NOT EXISTS
SELECT s.StudentID, s.Name
FROM Student s
WHERE NOT EXISTS (
    SELECT c.CourseID
    FROM Course c
    WHERE c.DeptID = 10
      AND NOT EXISTS (
          SELECT 1 FROM Enrollment e
          WHERE e.StudentID = s.StudentID AND e.CourseID = c.CourseID
      )
);

-- Lab 6.1 (Method B): Relational Division via GROUP BY + HAVING COUNT
SELECT e.StudentID
FROM Enrollment e
JOIN Course c ON e.CourseID = c.CourseID
WHERE c.DeptID = 10
GROUP BY e.StudentID
HAVING COUNT(DISTINCT e.CourseID) = (SELECT COUNT(*) FROM Course WHERE DeptID = 10);

-- Lab 6.2: Finding Maximum CGPA Without MAX() (Mirroring Relational Algebra)
SELECT DISTINCT CGPA FROM Student
EXCEPT
SELECT s1.CGPA
FROM Student s1
JOIN Student s2 ON s1.CGPA < s2.CGPA;
```

Solutions to Part H: "Spot the Mistake" Diagnostic Challenges (6.1 to 6.3)

Diagnostic Solution 6.1 | Projection Does Not Distribute Over Set Difference

Writing π_{DeptID}(Student − Alumni) = π_{DeptID}(Student) − π_{DeptID}(Alumni) is a classic algebraic fallacy. If Dept 10 has both current Students and Alumni, π_{DeptID}(Student) − π_{DeptID}(Alumni) removes Dept 10 completely, whereas π_{DeptID}(Student − Alumni) rightly keeps Dept 10!

Diagnostic Solution 6.2 | Using AND Instead of Intersection (∩) on a Single Column

Writing σ_{CourseID="CS301" ∧ CourseID="CS201"}(Enrollment) ALWAYS returns 0 rows (∅), because in any single tuple of Enrollment, the atomic attribute CourseID cannot simultaneously equal "CS301" and "CS201"! To find students enrolled in BOTH courses, you must intersect two projections:
π_{StudentID}(σ_{CourseID="CS301"}(Enrollment)) ∩ π_{StudentID}(σ_{CourseID="CS201"}(Enrollment))!

Diagnostic Solution 6.3 | Natural Join with Zero Common Attributes Is R × S, Not ∅

When R(A,B) and S(C,D) share no attribute names, the vacuous equality condition on common attributes is trivially satisfied for every pair of tuples, so R ⋈ S = R × S (|R| × |S| rows, never 0 rows unless R or S is empty).

END OF CHAPTER 6 | Up Next: Chapter 7 — SQL Fundamentals: DDL, DML, Constraints, Aggregates, and NULL Logic

In Chapter 7, we transition from formal Relational Algebra and Calculus to production SQL—mastering the 8-stage Logical Query Execution Order, DDL/DML, Pattern Matching, Set vs. Bag operators, the 7 Deadly SQL NULL Traps, and GROUP BY / HAVING / Conditional Aggregation!